AI Stock Research Dashboard - Interview Cheat Sheet

The pitch (say this first)

Full-stack stock research app: a Streamlit UI calls a FastAPI backend that pulls market data (yfinance), compares the stock to its sector ETF, asks an LLM (Claude/OpenAI, chosen by config) for an analyst note, and caches results in PostgreSQL. Containerized with Docker, tested in CI (GitHub Actions), deployable to Kubernetes or AWS via Terraform.

Request lifecycle (know this cold)

1. Browser -> Streamlit -> HTTP GET /api/stock/NVDA (FastAPI).
2. Check PostgreSQL cache; if fresh (<1h) return instantly (cached) - no fetch, no LLM cost.
3. Miss -> yfinance: price (fast_info) + sector/name (.info) + 1-month history + news.
4. pandas computes the 1-month return; map sector -> ETF, get ETF return -> alpha.
5. AI router builds a prompt -> Claude / OpenAI / mock (by config), with a timeout.
6. Save result to Postgres (best-effort) -> return JSON.
7. Streamlit renders the chart, metrics, news and AI note.

Tech stack (one line each)

Tech	Why it is here
FastAPI	Typed HTTP API; one endpoint orchestrates the whole flow.
Streamlit	Python -> web UI (chart, metrics, news, AI text).
yfinance + pandas	Free market data + the return maths.
OpenAI / Claude router	Provider chosen by env var; deterministic mock fallback.
PostgreSQL + SQLAlchemy	Cache - an optimization, not a hard dependency.
Docker + Compose	One command runs DB + backend + frontend, with health gating.
pytest + ruff + Actions	Tests, lint, image build, IaC validation on every push.
Kubernetes (kind)	Run the containers in a real cluster, locally.
AWS + Terraform	ECR, ECS Fargate, RDS, Secrets Manager, ALB - as code.

Top interview answers

Q: Where's the slow part?
A: yfinance + the LLM. Cached 1h in Postgres, timeouts on LLM calls, price from the fast endpoint.

Q: What if the DB is down?
A: Caching turns off; the app keeps serving live results (graceful degradation).

Q: How does it scale?
A: Stateless services + all state in Postgres -> run more pods/tasks behind a load balancer; an HPA scales the backend on CPU.

Q: How are secrets handled?
A: .env (gitignored) locally / Kubernetes Secrets / AWS Secrets Manager injected at runtime. Never in code or images.

Q: CI vs CD here?
A: CI runs lint, tests, docker build, terraform validate, kubeconform. Deployment is a manual Terraform step.

Q: Add a 3rd AI provider?
A: Write one adapter function + a branch in the router. Nothing else changes - that's the point of the abstraction.

Honest trade-offs (sound senior)

- create_all, not Alembic migrations - fine for one table; Alembic in production.
- Sync endpoints in a threadpool - timeouts protect against slow upstreams; async if high-throughput.
- No auth/rate-limit yet - add before exposing the (cost-incurring) endpoint publicly.

Three lines that land

"The cache is an optimization, not a hard dependency."
"Stateless services + state in Postgres = horizontal scaling."
"I wrote and validated the IaC (CI runs terraform validate + kubeconform) and ran it on kind; AWS is deploy-ready."